

Lab Week 4 – Replication Testing

23378247 – Jakub Bolka

GitHub: <https://github.com/SpencerLabAQ/performance-test-coverage>

Paper Research: <https://link.springer.com/article/10.1007/s10664-025-10712-3>

Excel Tracker for the Project: https://ulcampus-my.sharepoint.com/:x/g/personal/abdul_razzaq_ul_ie/IQDA_I7-eJ46Sa8pR9pObwzfAawJzwu0Uig4vBFZV3kGVXY?e=mgdefl

Week 02 + 03

- Pick group members.
- RQ1: Can we replicate the paper's results on performance-test coverage using the provided GitHub code and data?
- RQ2: Does adding new code metrics (or using a different machine learning model) improve how accurately we predict performance-test coverage?

Week 04

Freezing means locking in the initial plan/answers before later results are known, so the team cannot retrospectively change the original prediction.

Replication Environment

Operating system	Windows 11
Python	3.10.11 in a virtual environment
Repository branch	main
Dependency issue	requirements.txt pinned numpy==1.24.2, while imbalanced-learn==0.13.0 requires numpy>=1.24.3
Resolution	Minimal change to numpy==1.24.3; all remaining pinned packages retained

Paper context: The study evaluates JMH performance benchmarks and JUnit functional tests at method level. For RQ1, it compares overall coverage, direct coverage and overlap between tests.

Test 1 – 1-Overlap-JMH.py

Command: cd analysis → python 1-Overlap-JMH.py

The script executed successfully and printed overlap ratios for all 28 JMh projects.

Selected results: byte-buddy 1.00, fastjson 0.91, commons-bcel 0.09, SquidLib 0.80.

Project Name	Overlap Ratio
0 arrow	0.65
1 byte-buddy	1.00
2 cantaloupe	0.90
3 client_java	0.73
4 commons-bcel	0.09
5 crate	0.78
6 eclipse-collections	0.85
7 fastjson	0.91
8 feign	0.87
9 HdrHistogram	0.77
10 imglib2	0.89
11 iri	0.33
12 jdbi	0.97
13 jetty.project	0.53
14 jgrapht	0.70
15 jooby	0.89
16 kafka	0.67
17 logbook	0.86
18 netty	0.73
19 objenesis	0.79
20 panda	0.48
21 protostuff	0.98
22 r2dbc-h2	0.98
23 rdf4j	0.97
24 RxJava	0.83
25 SquidLib	0.80
26 tinkerpops	0.82
27 vert.x	0.75

Project	Replicated JMh	Paper / Fig. 5	Comparison
byte-buddy	1.00	1.00	Match
fastjson	0.91	0.91	Match
commons-bcel	0.09	0.09	Match
SquidLib	0.80	≈0.80	Consistent
Status		PASSED	
Conclusion		The reproduced JMh overlap values match the paper for the selected projects. This supports successful replication of the JMh overlap analysis.	

Test 2 – 2-Overlap-JUnit.py

Command: python 2-Overlap-JUnit.py

The script took longer than Test 1 but completed successfully. It printed overlap ratios for the 19 projects with JUnit data.

Selected results: byte-buddy 0.70, fastjson 0.15, commons-bcel 0.66, SquidLib 0.50.

Project Name Overlap Ratio		
0	arrow	0.77
1	byte-buddy	0.70
2	cantaloupe	0.81
3	client_java	0.70
4	commons-bcel	0.66
5	fastjson	0.15
6	feign	0.77
7	HdrHistogram	0.78
8	imglib2	0.67
9	iri	0.65
10	jdbi	0.83
11	jgrapht	0.80
12	jooby	0.55
13	logbook	0.78
14	objenesis	0.57
15	panda	0.76
16	r2dbc-h2	0.59
17	SquidLib	0.50
18	vert.x	0.74

Project	JMH	JUnit	Observed difference	Paper consistency
byte-buddy	1.00	0.70	JMH higher	Consistent with Fig. 8
fastjson	0.91	0.15	JMH much higher	Exact values discussed
commons-bcel	0.09	0.66	JUnit higher	Exact values discussed
SquidLib	0.80	0.50	JMH higher	Consistent with Fig. 8

Paper comparison: The paper reports that JMH overlap is higher than JUnit overlap in 68% of projects. It explicitly highlights commons-bcel (0.09 JMH vs 0.66 JUnit) and fastjson (0.91 JMH vs 0.15 JUnit), which match this replication.

Status	PASSED
Conclusion	The JUnit overlap script completed successfully and the selected values are consistent with the paper. No manual interruption or code change was required.

Test 3 – 3-Coverage-JMH.py

Command: python 3-Coverage-JMH.py

The script completed without terminal output because it writes the analysis directly to outputs/coverageJMH.csv.

```
coverageJMH.csv X
outputs > coverageJMH.csv
1 Project Name,Total Methods,Methods Covered,Methods Directly Covered,Coverage Per
2 HdrHistogram,780,120,29,15.38,3.72,12,0.77,491,83,40.92
3 RxJava,8282,408,116,4.93,1.4,217,0.83,6538,2027,30.13
4 SquidLib,5663,441,194,7.79,3.43,236,0.8,10509,4891,44.53
5 arrow,3789,342,110,9.03,2.9,34,0.65,2532,328,74.47
6 byte-buddy,5046,107,23,2.12,0.46,39,1.0,1480,276,37.95
7 cantaloupe,3475,288,81,8.29,2.33,103,0.9,3954,675,38.39
8 client_java,386,22,16,5.7,4.15,33,0.73,247,185,7.48
9 commons-bcel,3132,396,106,12.64,3.38,3,0.09,434,111,144.67
10 crate,24155,1668,216,6.91,0.89,39,0.78,8525,636,218.59
11 eclipse-collections,15219,679,241,4.46,1.58,515,0.85,15779,3353,30.64
12 fastjson,17524,64,3,0.37,0.02,4,0.91,122,6,30.5
13 feign,979,121,23,12.36,2.35,8,0.87,548,54,68.5
14 imglib2,3432,251,59,7.31,1.72,25,0.89,1594,149,63.76
15 iri,1554,15,9,0.97,0.58,3,0.33,21,15,7.0
16 jdbi,2379,379,57,15.93,2.4,76,0.97,5255,436,69.14
17 jetty.project,18060,205,89,1.14,0.49,48,0.53,540,181,11.25
18 jgraph,3782,371,74,9.81,1.96,51,0.7,2206,407,43.25
19 jooby,3331,9,3,0.27,0.09,3,0.89,21,9,7.0
20 kafka,16157,485,194,3.0,1.2,27,0.67,1466,479,54.3
21 logbook,811,120,50,14.8,6.17,20,0.86,475,220,23.75
22 netty,16615,1671,442,10.06,2.66,221,0.73,23989,5147,108.55
23 objenesis,207,18,10,8.7,4.83,13,0.79,79,32,6.08
24 panda,1479,722,21,48.82,1.42,4,0.48,1167,46,291.75
25 protostuff,3312,41,9,1.24,0.27,16,0.98,328,62,20.5
26 r2dbc-h2,291,43,16,14.78,5.5,8,0.98,164,67,20.5
27 rdf4j,14041,1164,40,8.29,0.28,14,0.97,4033,79,288.07
```

Project	Coverage %	Direct %	Benchmarks	Overlap	Avg. coverage
byte-buddy	2.12	0.46	39	1.00	37.95
fastjson	0.37	0.02	4	0.91	30.50
commons-bcel	12.64	3.38	3	0.09	144.67
SquidLib	7.79	3.43	236	0.80	44.53

Paper context: Across all 28 projects, the paper reports average JMH method coverage of 8.8% and average direct coverage of about 2.1%. The selected projects show the same project-level values produced by the supplied replication data.

Status	PASSED
Conclusion	The script successfully generated coverageJMH.csv with the expected coverage, direct coverage, benchmark-count and overlap fields.

Test 4 – 4-Coverage-JUnit.py

Command: python 4-Coverage-JUnit.py

The script completed successfully without printing to the terminal. It generated outputs/coverageJUnit.csv.

```
Project Name,Total Methods,Methods Covered,Methods Directly Covered,Coverage Percentage,Direct Coverage Percentage,Total Tests,Overlap Ratio,Methods Tested,Methods Directly Tested,Average Coverage
HdrHistogram,780,551,188,70.64,24.1,147,0.78,4686,512,31.88
SquidLib,5663,351,173,6.2,3.05,73,0.5,862,278,11.81
arrow,3789,1881,952,49.64,25.13,869,0.77,44162,5630,50.82
byte-buddy,5046,646,486,12.8,9.63,6357,0.7,19361,4677,3.05
cantaloupe,3475,1521,974,43.77,28.03,3063,0.81,28944,5560,9.45
client_java,386,197,113,51.04,29.27,217,0.7,1462,464,6.74
commons-bcel,3132,1087,133,34.71,4.25,137,0.66,4581,153,33.44
fastjson,17524,6053,4879,34.54,27.84,4979,0.15,65302,4984,13.12
feign,979,440,213,44.94,21.76,913,0.77,17876,1920,19.58
imglib2,3432,1446,649,42.13,18.91,635,0.67,7270,1455,11.45
iri,1554,492,287,31.66,18.47,398,0.65,3126,860,7.85
jdbc,2379,982,310,41.28,13.03,1428,0.83,68684,5180,48.1
jgrapht,3782,2201,1043,58.2,27.58,2416,0.8,34452,4739,14.26
jooby,3331,1096,320,32.9,9.61,485,0.55,12474,882,25.72
logbook,811,434,214,53.51,26.39,564,0.78,3677,794,6.52
objenesis,207,62,29,29.95,14.01,45,0.57,136,52,3.02
panda,1479,877,69,59.3,4.67,61,0.76,3910,135,64.1
r2dbc-h2,291,138,95,47.42,32.65,259,0.59,613,360,2.37
vert.x,5685,2236,1099,39.33,19.33,4095,0.74,24438,5782,5.97]
```

Status	PASSED
Conclusion	The expected JUnit coverage CSV was created successfully, confirming completion of the fourth analysis script.

Script File Status Audit

Script File	Status	Findings	Action Taken
1-Overlap-JMH.py	Passed	Selected overlap figures match the paper.	Replicated successfully.
2-Overlap-JUnit.py	Passed	Completed successfully; selected values agree with the paper.	No change required.
3-Coverage-JMH.py	Passed	Generated coverageJMH.csv; no terminal output by design.	Verified generated CSV.
4-Coverage-JUnit.py	Passed	Generated coverageJUnit.csv and completed cleanly.	Verified generated CSV.

Spreadsheet Column Fill-Ins

2. Bounded sample / case IDs

Four target open-source repositories: byte-buddy, fastjson, commons-bcel and SquidLib.

3. Method / metric or decision rule

Execute the four original Python analysis scripts (1-Overlap-JMH.py through 4-Coverage-JUnit.py). Compare overlap-ratio outputs with the paper and verify that the coverage scripts generate the expected CSV outputs.

4. Expected result before execution

All four analysis scripts should run successfully on the supplied replication data, and the selected overlap values should reproduce the published results.

5. Reproduction result (brief)

Successful reproduction with one environment adjustment. All four analysis scripts completed. JMH and JUnit overlap values for the four selected projects were reproduced, and both coverage CSV files were generated.

6. Evidence link / commit / output

Console output for 1-Overlap-JMH.py and 2-Overlap-JUnit.py; outputs/Overlap_Ratios_JMH.csv; outputs/Overlap_Ratios_Junit.csv; outputs/coverageJMH.csv; outputs/coverageJUnit.csv; screenshots included in this document.

7. Event A prediction before execution

Prediction: the supplied replication package should execute on a clean Python 3.10 environment using the repository requirements and reproduce the paper's analysis outputs.

8. Event A result / decision

9. Concrete design problem

The repository's pinned dependency set is internally inconsistent. This prevents a clean 'pip install -r requirements.txt' on a fresh environment and weakens out-of-the-box reproducibility.

10. Alternative(s) considered

Option A: minimally raise NumPy from 1.24.2 to 1.24.3.

Option B: downgrade imbalanced-learn to a release compatible with NumPy 1.24.2.

Option C: bypass dependency resolution / use --no-deps, which risks an unstable environment.

11. Chosen intervention / no-change decision

Chosen intervention: keep Python 3.10.11 and all existing project pins, but change NumPy to 1.24.3 so that imbalanced-learn 0.13.0 can resolve normally. This is the smallest change and preserves the intended environment as closely as possible.

Overall Conclusion

Outcome: Successful replication of the Week 4 analysis tasks. All four scripts completed on the fresh Windows setup after resolving the NumPy dependency conflict. The JMH/JUnit overlap outputs for byte-buddy, fastjson, commons-bcel and SquidLib are consistent with the published study, and the coverage scripts generated the expected CSV files. The main reproducibility issue identified was dependency management rather than the analysis logic.